

Milestone 2: Scientific Contextualization and Tradeoff Analysis

Loop Unrolling and Controller Delay in High-Level Synthesis

Christian Percival
Hochschule Hamm-Lippstadt
Electronic Engineering

I. MILESTONE 2: SCIENTIFIC CONTEXTUALIZATION AND TRADEOFF ANALYSIS

A. *Scientific Context*

- Loop unrolling is a well-known optimization in compiler design.
- In traditional software compilation, it is mainly used to:
 - reduce loop overhead,
 - expose instruction-level parallelism,
 - improve scheduling opportunities,
 - increase performance on processors.
- In high-level synthesis, the same optimization has different consequences.
- The unrolled loop body is not only compiled into more instructions, but can become more hardware structure.
- This means that loop unrolling affects:
 - latency,
 - datapath size,
 - controller complexity,
 - multiplexer complexity,
 - resource usage,
 - timing closure.
- The reference paper is positioned between compiler optimization and hardware synthesis.
- Its main contribution is that it studies loop unrolling from the perspective of controller delay in HLS.

B. *Related Approaches*

- Classical loop unrolling:
 - Focuses on reducing loop overhead and improving software performance.
 - Usually considers processor execution, instruction scheduling, and memory behavior.
 - Does not directly consider FSM delay because software does not generate a hardware controller in the same way.
- High-level synthesis scheduling:
 - Decides which operations are executed in which clock cycles.

- Determines the number of control steps.
- Affects both latency and controller complexity.

- Loop pipelining:

- Allows new loop iterations to start before previous iterations are complete.
- Often improves throughput.
- Can interact with loop unrolling.
- May require additional hardware resources and careful dependency handling.

- Resource sharing:

- Reuses functional units such as adders and multipliers across multiple operations.
- Reduces area.
- Can increase multiplexer and controller complexity.

- Exhaustive design-space exploration:

- Synthesizes many possible design versions.
- Can find good solutions.
- Becomes expensive when many loops and unroll factors are involved.

- Estimation-based design-space exploration:

- Uses high-level models to estimate performance or delay.
- Avoids synthesizing every possible design.
- Is faster but less exact than full synthesis.

C. *Main Tradeoff*

- The central tradeoff is between latency reduction and controller-delay increase.
- Small unroll factors:
 - keep the controller simpler,
 - use fewer hardware resources,
 - may still have relatively high latency.
- Medium unroll factors:
 - often give a good latency improvement,
 - usually avoid the worst timing and area penalties,
 - may be the most practical choice.
- Large or full unroll factors:
 - expose maximum parallelism,

- reduce the number of loop iterations,
- can greatly increase hardware resources,
- can increase FSM and multiplexer complexity,
- can create timing problems.

D. Latency Versus Clock Period

- A design with fewer cycles is not automatically faster.
- Total execution time depends on both cycle count and clock period.
- Loop unrolling can reduce the number of cycles.
- However, if unrolling increases the critical path, the clock period may need to become longer.
- Therefore, the final performance depends on:

$$\text{Execution Time} = \text{Latency in Cycles} \times \text{Clock Period} \quad (1)$$

- This means that a design with lower latency in cycles may still be worse if it has a much slower clock.

E. Resource Tradeoffs

- Loop unrolling can increase resource usage because more operations are present in the loop body.
- More arithmetic operations may require more:
 - adders,
 - multipliers,
 - DSP blocks,
 - registers,
 - multiplexers,
 - memory ports.
- If enough resources are available, unrolling can expose parallelism.
- If resources are limited, unrolled operations may still need to share the same hardware units.
- Resource sharing can reduce area but increase controller and multiplexer complexity.

F. Timing Tradeoffs

- Unrolling can make the controller more complex.
- A more complex controller may have longer combinational logic paths.
- Larger multiplexers can also increase datapath delay.
- These effects can reduce the maximum clock frequency.
- Timing closure becomes more difficult when the unroll factor is too large.

G. Scalability Tradeoffs

- The design-space exploration problem grows when there are multiple loops.
- Each loop can have several possible unroll factors.
- Exhaustively trying every combination can become impractical.
- The reference paper addresses this by:
 - pruning the search space,
 - considering promising unroll factors,
 - using a priority function,

- estimating delay instead of synthesizing every candidate.

H. Comparison of Design Choices

TABLE I
TRADEOFF SUMMARY FOR LOOP UNROLLING IN HLS

Design Choice	Advantages	Disadvantages
No unrolling	Simple controller and low resource usage	Higher loop latency
Partial unrolling	Good balance between latency and complexity	Requires choosing a suitable factor
Full unrolling	Maximum exposed parallelism	High resource usage and possible timing problems
Exhaustive exploration	Can find strong design candidates	Very high synthesis time
Estimation-based exploration	Faster and more scalable	Less exact than full synthesis

I. Relevance for Embedded Systems

- Embedded systems often have strict timing, area, power, and cost constraints.
- HLS is useful for embedded hardware accelerators, but optimization choices must be hardware-aware.
- Loop unrolling can improve performance, but may increase area and timing pressure.
- A balanced unroll factor is important for practical embedded hardware.
- The paper is relevant because it shows that controller delay must be considered, not only datapath latency.

J. References and Related Work Situation

- Related work exists in compiler optimization, loop unrolling, software pipelining, HLS scheduling, timing estimation, and design-space exploration.
- However, the specific focus on controller delay caused by loop unrolling in HLS is more specialized.
- Therefore, not every loop-unrolling paper is equally relevant.
- The strongest related work should be selected from:
 - loop unrolling in compilers,
 - HLS timing estimation,
 - HLS scheduling,
 - loop pipelining,
 - design-space exploration.

REFERENCES

- [1] S. Kurra, N. K. Singh, and P. R. Panda, “The Impact of Loop Unrolling on Controller Delay in High Level Synthesis,” DATE, 2007.